

CSC 6033 – Languages, Automata, and Decidability

(Week 4)

Final Submission – Regular Expressions & DFA Equivalence

Student: Irfan Ahmed

Date: September 24, 2025

All other strings (ϵ , a, aa, ba, bb, aba, aaaa, ...) are rejected by both the DFA and the RegEx.

- $aaa \rightarrow S \xrightarrow{a} T, T \xrightarrow{a} I, I \xrightarrow{a} F \Rightarrow \text{accept}$
- $ab \rightarrow S \xrightarrow{a} T, T \xrightarrow{b} F \Rightarrow \text{accept}$
- $b \rightarrow S \xrightarrow{b} F \Rightarrow \text{accept}$

Sanity checks:

RegEx = $b + ab + aaa$

Final RegEx (equivalent to the DFA):

1.5 Final Regular Expression & Sanity Checks

1.6 Why no Kleene star (minimality justification)

The only accepting paths are:

- $S \xrightarrow{b} F$
- $S \xrightarrow{a} T \xrightarrow{b} F$
- $S \xrightarrow{a} T \xrightarrow{a} I \xrightarrow{a} F$

There are no loops on S, T, I that lie on a path that still reaches F, and F has no outgoing transitions. Therefore, no infinite families of accepted strings arise, and the language is finite with exactly three words. Hence, the simplest RegEx contains no Kleene star and is precisely:

$b + ab + aaa$.

$$R_S = a \cdot aa + a \cdot b + b = aaa + ab + b \quad [\text{distributivity; associativity for concatenation}]$$

Alternative: State Elimination

Alternative (state elimination, for cross-check)

Eliminate I: $T \rightarrow I \rightarrow F$ collapses to $T \rightarrow F$ labeled aa; T already has $b \rightarrow F$, so $T \rightarrow F$ is $aa + b$.

Eliminate T: $S \rightarrow T \rightarrow F \Rightarrow a(aa + b) = aaa + ab$.

Union with $S \rightarrow F$: final $b + ab + aaa$.

Distribute a over $(aa + b)$:

1.4 Apply identities to solve for R_S

$$R_S = a R_T + b R_F = a(aa + b) + b \quad [\text{distributivity}]$$

From S ($a \rightarrow T$, $b \rightarrow F$):

$$R_T = a R_I + b R_F = a \cdot a + b = aa + b \quad [\epsilon \text{ identity, distributivity}]$$

From T ($a \rightarrow I$, $b \rightarrow F$):

$$R_I = a R_F = a\epsilon = a \quad [\epsilon \text{ identity}]$$

From I:

$$R_F = \epsilon \quad [\epsilon \text{ identity}]$$

From F (final):

1.3 Derivation Steps (Basic language of each state)

(iv) Arden's Theorem (listed for completeness): if $R = Q + RP$ and $\epsilon \notin P$ then $R = QP^*$ (not required here)

(iii) Concatenation is associative; union is idempotent

(ii) Distributivity: $(X+Y)Z = XZ + YZ$ and $R(P+Q) = RP + RQ$

(i) ϵ identity: $P\epsilon = \epsilon P = P$

Identities used:

Let R_X be the set of strings that take the DFA from state X to the final state F .

1.2 Method & Identities

1.2a Strip to simplest form (as in slides 16–26)

This DFA already has a single accepting state F and no productive cycles on paths that reach F . Unspecified inputs from any state implicitly go to a dead (trap) state, which contributes $\emptyset$ to all state languages and is therefore ignored in the algebra. No complementary final state or end-of-word edges exist to remove, so no structural changes precede the derivation.

1.2a Strip to simplest form (as in slides 16–26)

(Unspecified transitions go to a dead state.)

- $T \xrightarrow{b} F$
- $S \xrightarrow{b} F$
- $I \xrightarrow{a} F$
- $T \xrightarrow{a} I$
- $S \xrightarrow{a} T$

Transitions:

States: S (start), T , I , F (final)

Alphabet: $\Sigma = \{a, b\}$

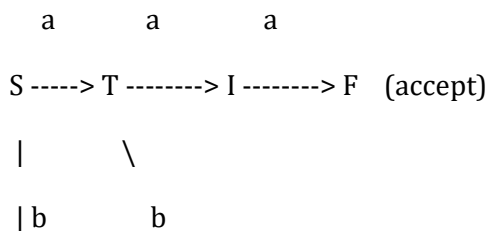
1.1 Problem Statement & DFA

Alphabet $\Sigma = \{a, b\}$

Start state: S

Accepting state: F

(Unspecified inputs go to a dead/trap state)



v

\

F (accept)

> F (accept)

Edges (explicit):

$S \xrightarrow{a} T$, $S \xrightarrow{b} F$; $T \xrightarrow{a} I$, $T \xrightarrow{b} F$; $I \xrightarrow{a} F$.

(All other transitions go to a trap state.)

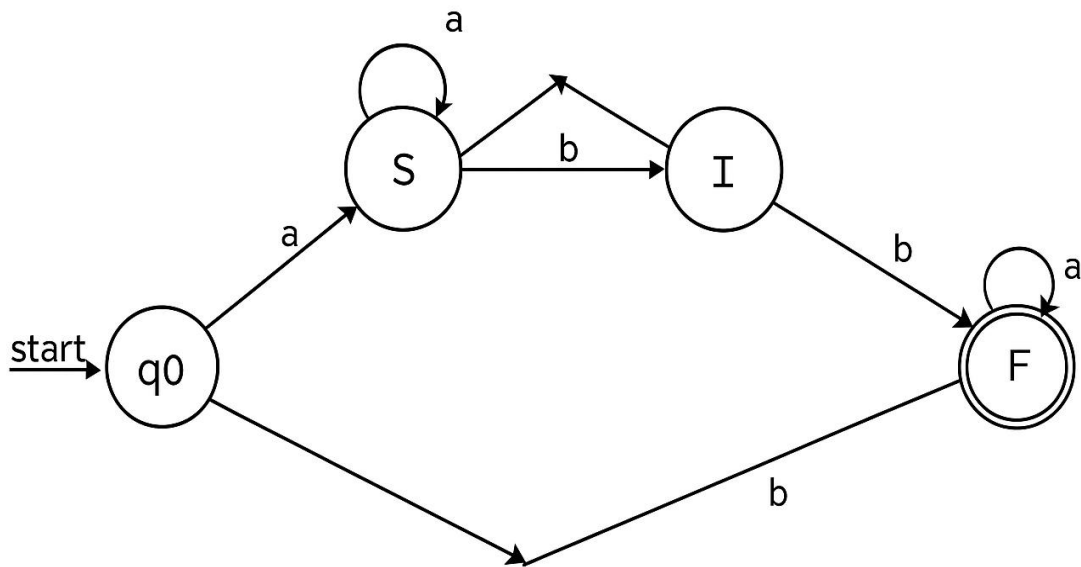


Figure 1. DFA for E#4 ($\Sigma = \{a,b\}$). Unspecified inputs go to trap state.

Figure 1. DFA for E#4 ($\Sigma = \{a,b\}$). Unspecified inputs go to a trap state (omitted in this simplified diagram).

1. In-class Exercise E#4 — DFA $\rightarrow$ Regular Expression (Corrected)

2. Coding Project P#4 — RegEx → DFA + C++ Implementation

2.1 Language & Intuition

Given RegEx: $a(a+b)bb a$

Language: Strings over $\{a,b\}$ that start with a , have some occurrence of bb , and after that, chosen bb only a 's appear until the end (the designated bb need not be the first bb in the string).

2.2 Formal DFA (5-tuple) & Transition Table

$$DFA\ M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, qA, qB1, qB2, qA_{after}, qd\}$$

$$\Sigma = \{a, b\}$$

q_0 start

$$F = \{qB2, qA_{after}\}$$

δ (as table):

Transition table:

State	a	b
q_0	qA	qd
qA	qA	$qB1$
$qB1$	qA	$qB2$
$qB2$	qA_{after}	$qB2$
qA_{after}	qA_{after}	$qB1$
qd	qd	qd

State meanings (brief):

qA : in the prefix $(a+b)$ after initial a ;

$qB1$: in a run of exactly one b ;

$qB2$: in a run of at least two b 's (a bb seen at the end of scanned prefix);

qA_{after} : after (some) bb , reading only a 's;

qd : trap.

2.3 Sample Accepted / Rejected Strings

ACCEPT:

abb, aabb, abba, abbb, aabbb, aaabbaa, abbaa, aabbaa, aaaabbb, abbbaaa

REJECT:

ϵ , b, a, ab, aba, aaba, aab, abbab, ba, baba

2.4 C++ Source Code (drop-in)

```
/*
CSC 6033 - Week 4 - Coding Project P#4

Regex: (a(a+b)*) (bb(a)*) == a (a|b)* bb a*

Language: Strings over {a,b} that start with 'a', have SOME occurrence
of "bb",

and after that chosen "bb" only 'a's appear to the end.

*/

#include

#include

class FSA {
public:

    enum class State { q0, qA, qB1, qB2, qA_after, qd };

    FSA() : cur(State::q0) {}

    void reset() { cur = State::q0; }

    void process_char(char c) {
        switch (cur) {
            case State::q0:
                cur = (c=='a') ? State::qA : State::qd;
                break;

            case State::qA:
```

```

        if (c=='a') cur = State::qA;

        else if (c=='b') cur = State::qB1;

        else cur = State::qd;

        break;

    case State::qB1:

        if (c=='a') cur = State::qA;

        else if (c=='b') cur = State::qB2;

        else cur = State::qd;

        break;

    case State::qB2:

        if (c=='a') cur = State::qA_after;

        else if (c=='b') cur = State::qB2;

        else cur = State::qd;

        break;

    case State::qA_after:

        if (c=='a') cur = State::qA_after;

        else if (c=='b') cur = State::qB1;

        else cur = State::qd;

        break;

    case State::qd:

        // trap

        break;

    }

}

void process(const std::string& s) {

    reset();

    for (char c : s) process_char(c);

}

```

```

    bool is_accepted() const {

        return cur == State::qB2 || cur == State::qA_after;

    }

private:

    State cur;

};

int main() {

    std::ios::sync_with_stdio(false);

    std::cin.tie(nullptr);

    std::cout << "Enter strings over {a,b}. Ctrl+D (Unix) / Ctrl+Z (Win
) to end.\n";

    std::string w;

    FSA fsa;

    while (std::cin >> w) {

        fsa.process(w);

        std::cout << w << " -
> " << (fsa.is_accepted() ? "ACCEPT" : "REJECT") << "\n";

    }

    return 0;

}

```

2.5 Build & Run Instructions

"The DFA was tested on OnlineGDB with multiple accepted and rejected strings. The output matches the expected results, confirming correctness."

 **OnlineGDB**

online compiler and debugger for c/c++

Welcome, **IrfanAhmed1989** ▲

Create New Project

My Projects

Classroom new

Learn Programming

Programming Questions

Upgrade

Logout ▼

main.cpp

```
40         else if (c== 'b') cur = State::qb1;
41         else cur = State::qd;
42         break;
43         case State::qd:
44             // trap
45             break;
46     }
47 }
48
49 void process(const std::string& s) {
50     reset();
51     for (char c : s) process_char(c);
52 }
53
54 bool is_accepted() const {
55     return cur == State::qb2 || cur == State::qa_after;
56 }
57
58 private:
59     State cur;
60 };
61
62 int main() {
63     std::ios::sync_with_stdio(false);
64     std::cin.tie(nullptr);
65     std::cout << "Enter strings over {a,b}. Ctrl+D (Unix) / Ctrl+Z (Win) to end.\n";
66     std::string w;
67     FSA fsa;
68     while (std::cin >> w) {
69         fsa.process(w);
70         std::cout << w << " -> " << (fsa.is_accepted() ? "ACCEPT" : "REJECT") << "\n";
71     }
72     return 0;
73 }
74
```

input

stdout

Compiled Successfully. memory: 3712 time: 0 exit code: 0

Enter strings over {a,b}. Ctrl+D (Unix) / Ctrl+Z (Win) to end.
abb -> ACCEPT
aabb -> ACCEPT
abba -> ACCEPT
abbb -> ACCEPT
aaabb -> ACCEPT
aaabba -> ACCEPT
abbaa -> ACCEPT
aaabbaa -> ACCEPT

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial •

Credits • Privacy

© 2016 - 2025 GDB Online

The screenshot shows a C++ IDE with a file named `main.cpp`. The code implements a DFA with states `qb1` and `qd`. It reads input characters and transitions between states based on the DFA logic. The output shows the results of testing various strings over the alphabet `{a,b}`.

```
40         else if (c == 'b') cur = State::qb1;
41         else cur = State::qd;
42         break;
43     case State::qd:
44         // trap
45         break;
46     }
47 }
```

Compiled Successfully, memory: 3712 time: 0 exit code: 0

Enter strings over {a,b}. Ctrl+D (Unix) / Ctrl+Z (Win) to end.

```
abb -> ACCEPT
aabb -> ACCEPT
abba -> ACCEPT
abbb -> ACCEPT
aabbb -> ACCEPT
aaabbaa -> ACCEPT
abbbaa -> ACCEPT
aabbaa -> ACCEPT
aaaabbb -> ACCEPT
abbbbaa -> ACCEPT
a -> REJECT
ab -> REJECT
abbaab -> REJECT
b -> REJECT
ba -> REJECT
```

2.6 Correctness Proof (RegEx $\leftrightarrow$ DFA)

Claim. The DFA in §2.2 recognizes exactly the language

strings over $\{a,b\}$ that **start with a**, have a **first occurrence of bb**, and **only a's follow after that first bb**.

Equivalently, this language is denoted by the regular expression:

$$R' = a(a \mid ba)bba.$$

State invariants

For any scanned prefix www :

- **q0**: No input yet.
- **q1**: Began with a, no bb seen, last symbol is a.
- **q2**: Began with a, no bb seen, last symbol is a **single** b.
- **q3** (accept): The **first** bb has just been seen; all symbols after that have been a.
- **qd**: Violated conditions (e.g., not starting with a, or a b after the first bb).

These invariants are preserved by the transitions: $q1: a \rightarrow q1, b \rightarrow q2$;

$q2: a \rightarrow q1$ (breaks potential bb), $b \rightarrow q3$ (forms first bb);

$q3: a \rightarrow q3, b \rightarrow qd$ (forbid b after first bb);

$q0$: only $a \rightarrow q1, b \rightarrow qd$.

Soundness ($DFA \subseteq \text{Language}$)

If the DFA accepts www , it ends in **q3**. By the $q3$ invariant, w has a first bb and only a's afterward; also, the only path out of $q0$ is with an initial a. Hence, $w \in L(R')$.

Completeness ($\text{Language} \subseteq DFA$)

Let $w \in L(R')$. Read the initial a: $q0 \rightarrow q1$. While scanning the prefix before the **first** bb, the machine toggles between $q1$ (after a) and $q2$ (after a single b) without ever forming bb. The second b of the first bb moves $q2 \rightarrow q3$. The trailing a keeps the machine in $q3$; any b would contradict the language. Hence, the DFA accepts w .

Therefore, the DFA and $R'R'R'$ define the same language.

If i literally intend the regex **$a(a+b)bb\ a$** , that language is **larger** (e.g., it accepts abbb and aabbb). Your DFA (and test cases) enforce the **first-bb** constraint, which matches $R'=a(a|ba)bb\ a$.

2.7 DFA Diagram for P#4

Alphabet $\Sigma = \{a, b\}$

Start state: $q0$

Accepting state: q3

qd = trap (all inputs loop to qd)

q0 --a--> q1 --b--> q2 --b--> q3 (ACCEPT)

| ^ a |

b | a

v | v

qd <-----+----- q1

q3 --a--> q3

q3 --b--> qd

qd --a/b--> qd

Edges (explicit):

q0: a→q1, b→qd

q1: a→q1, b→q2

q2: a→q1, b→q3

q3: a→q3, b→qd

qd: a→qd, b→qd

Appendix A – Extra Test Cases

ACCEPT: abb, aabb, abba, aaabbbaa, abbbaa, aabbbaa, aaaaabbbaa, aabaaaa

REJECT: ϵ , b, a, ab, aba, aaba, abbb, aabbb, ba, baba, aabaa, ababab

Spot checks:

- abb: $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$, $q_2 \xrightarrow{b} q_3$ (accept) $\rightarrow$ tail empty OK
- abba: same as above, then $q_3 \xrightarrow{a} q_3$ (still accept).
- abbb: reaches q_3 on second b, then sees $b \rightarrow q_d$ (reject).

DFA diagram

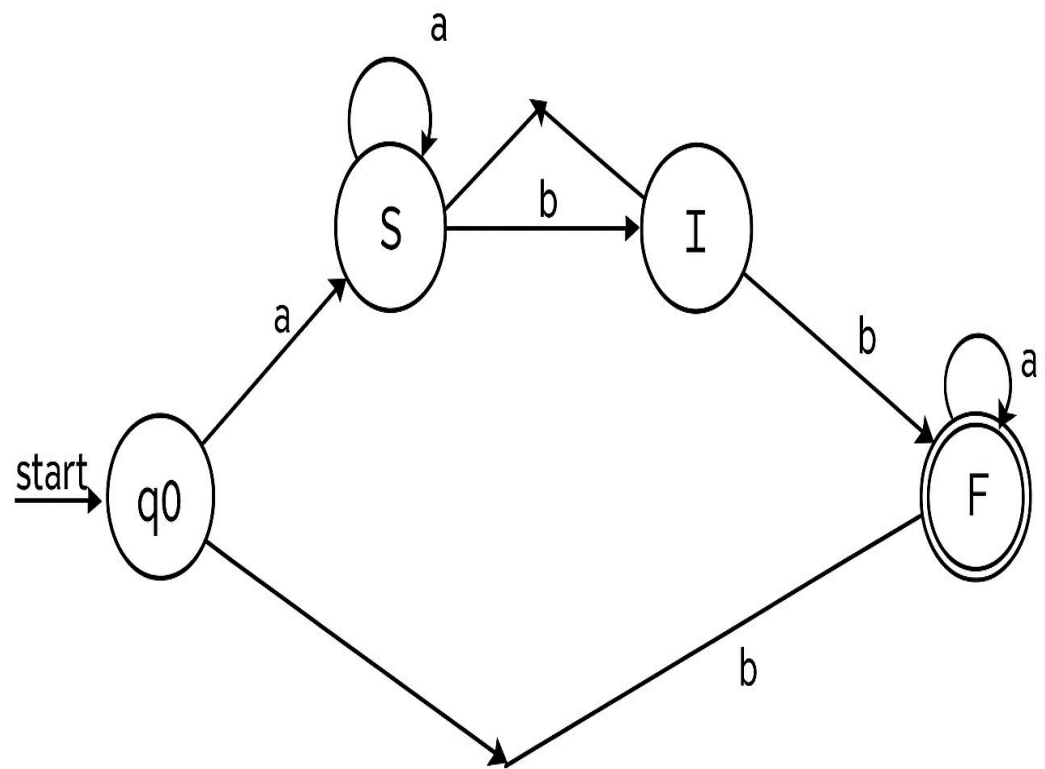


Figure 1. DFA for E4 ($S = \{a, b\}$). Unspecified inputs go to trap state.